Knowledge Tracing for Large Language Models

Project Workbook

By

Ronak Patel
Bhagyesh Rathi
Aniket Sarrin

October 1, 2026

Advisor: Dr. Mahima Suresh Agumbe

CMPE 295A, San José State University

Chapter 1
Literature Search, State of the Art

1.1 Literature Search

This project sits at the intersection of four literatures that have not previously been combined: knowledge tracing from educational data mining, psychometric evaluation of language models, the study of LLM training dynamics across checkpoints, and temporal item response theory from political science and learning analytics. The search covered arXiv (cs.CL, cs.LG, cs.AI), the ACL Anthology, OpenReview (ICLR 2025 and 2026, NeurIPS 2024 and 2025, ICML 2026), the EDM/ AIED/LAK proceedings, Semantic Scholar forward citations of the five closest papers, and GitHub and HuggingFace for released data and code. Roughly 60 queries and 80 forward citations were screened; about 45 papers were read in full. The search was carried out in two passes (September 22 and September 30, 2026) and the second pass was spot-verified against primary sources. Cross-references below are to the numbered entries in the References section.

Knowledge tracing

Knowledge tracing (KT) is the learner model inside intelligent tutoring systems. Bayesian Knowledge Tracing [1] models each skill as a two-state hidden Markov model (unmastered/mastered) with per-skill initial-mastery, learn, guess and slip probabilities, updated after every observed response. Khajah, Lindsey and Mozer [2] showed that adding forgetting to BKT removes about half of DKT's reported advantage on Assistments, and that other extensions (per-student ability, skill discovery) close more of it, which matters for us because our data has few learners. Deep Knowledge Tracing [3] replaced the HMM with a recurrent network, and later attention-based models such as AKT [4] and SAINT [45] added item and context representations; these are trained on large educational datasets with thousands of student sequences, and their behaviour with a handful of learners is not established. Yeung and Yeung [49] analysed consistency problems in DKT's predictions, and Vie and Kashima [5] showed that DKT is an implicit dynamic multidimensional IRT model, which gives the bridge between KT and item response theory that our method relies on. pyBKT [6] is the maintained open-source BKT implementation; its internal data format allows several observations per time step with a single transition per step, but its E-step multiplies raw emission probabilities across observations before normalising, which underflows for batches of hundreds of items, so a log-space implementation is needed for our data. DAS3H [7] is a learned non-latent baseline that combines item difficulty, skill and time-window counts in a logistic model; its practice-window features must be re-defined over training progress for our setting.

Psychometric evaluation of language models

Item response theory (IRT) treats each benchmark question as an item with a difficulty and a discrimination and each model as a respondent with a latent ability. Fluid Benchmarking [8] from AI2 fits 2PL IRT to Open LLM Leaderboard results and then selects items adaptively as a model improves during pretraining; its paper reports 467 checkpoints of six open models, and the public release we inspected contains 452 of them with the per-item, per-checkpoint correctness that forms our Phase 1 dataset. Item Response Scaling Laws [9] fits IRT to 6,612 pretraining checkpoints and 37,682 items from the DataDecide release [44], estimates each checkpoint's ability independently by adaptive testing, and fits a scaling law in log-compute through those estimates. It is the closest prior work on the psychometric side. It has no sequential state linking checkpoints, its item parameters are calibrated once and frozen, it has no per-skill decomposition, and it validates across model sizes and item subsets rather than by forecasting later checkpoints within a run; forgetting is not something its scaling law represents. Jiang et al. [10] study when IRT estimates for AI evaluation can be trusted: in their simulations 30 respondents were insufficient for item recovery and 100 were adequate (sizes in between were not tested), and rank recovery degraded sharply under skewed ability distributions, which a pretraining run has by construction. Zheng and Yang [11] show differential item functioning across model families on MMLU, HellaSwag and WinoGrande. Paquin and Jain [12] audit MMLU with a 2PL model over 1,000 models and find a latent correlation of 0.965 between STEM and non-STEM ability; they conclude the two are distinguishable (a two-dimensional model fits held-out responses better), which motivates testing how separable the 57 subjects are rather than assuming either answer. Beyond Scores [13] fits multidimensional IRT with a 35-dimension mathematics Q-matrix to 41 frozen models, and Laplace-PSN-IRT [14] puts an approximately calibrated posterior on LLM ability while its joint item posterior is poorly calibrated; both are static. Cognitive diagnosis has been applied to 30 frozen financial LLMs [15], and CDS [16] uses a per-skill diagnosis to synthesise training data for weak skills, which is the downstream application of our mastery estimates. ATLAS [17] releases calibrated item banks (primarily 3PL, with a 2PL comparison) over 3,000 models for five benchmarks, not including MMLU, which we can use to cross-check Fluid's calibration where items overlap. Signal and Noise [18] quantifies checkpoint-to-checkpoint evaluation noise on our exact benchmarks and shows that averaging intermediate checkpoints and filtering low-signal MMLU subtasks improve decision accuracy; it is the aggregate-level precedent that motivates our item-history baseline, though that baseline is inspired by rather than an implementation of their method.

LLM training dynamics across checkpoints

Pythia [19] and OLMo [20] made intermediate pretraining checkpoints public and started a literature on what models learn when. Mishra, Poesia and Goodman [21] introduced MathCAMPS, a synthetic math benchmark tagged with 44 Common Core standards, and showed that Pythia, OLMo and Amber acquire math skills in roughly the order schools teach them; they report only per-skill accuracy curves. Liu et al. [22] tracked 91 elemental and composite tasks across nine models and found that emergence order (defined by an accuracy threshold) is stable across models and that composite skills usually appear after their components; they also predict held-out composite-task trajectories from function vectors, and their repository releases per-item correctness for Amber, OLMo-2 and Pythia-6.9B. Chang et al. [23] injected fictional facts into OLMo pretraining and observed forgetting that follows a power law in training steps, which motivates testing non-constant or time-scaled forgetting in our models. Harmon et al. [24] count item-level correct-to-incorrect transitions to map forgetting during post-training, and Liao et al. [25] study entity forgetting during pretraining. Hu et al. [26] fit hidden Markov models to aggregate training metrics to find latent phases of training, so latent-state models of training do exist at the aggregate level; Michaelov, Levy and Bergen [27] show that behavioural phases are consistent across architectures. Dominguez-Olmedo, Dorner and Hardt [28] show that, under their evaluation, Pythia performs at chance on MMLU and that apparent differences between model families shrink after equal fine-tuning on task-relevant data, which we must address when interpreting our cross-model results. Among these, none fits a per-skill latent learner state to item-level checkpoint responses and evaluates it by forecasting item-level responses at later checkpoints within a training run: Hu et al. model aggregate metrics, and Liu et al. forecast task-level trajectories.

Temporal item response theory

Dynamic IRT models let ability evolve over time. Martin and Quinn [29] introduced a dynamic ideal-point model for the U.S. Supreme Court (29 justices, at most nine sitting at once, over 47 terms, with different cases each term), with a random-walk prior on ability estimated by MCMC using Kalman forward-filtering and backward-sampling. Imai, Lo and Olmsted [30] provide a fast variational EM implementation in the R package emIRT (function dynIRT). This is the closest structural precedent for few learners observed over many periods, but it is not a drop-in fit: dynIRT assigns each item to exactly one period, so sharing an item's difficulty and discrimination across all checkpoints (our setting, where the same 27,842 items are re-asked at every checkpoint) requires adapting the model, and its manual states that its variational variance estimates are far too small to use as uncertainty. T-SKIRT [31] from Knewton models ability as a Wiener process indexed by elapsed time with a

closed-form effective discrimination that down-weights old responses; its temporal-only ablation gains 2.2 accuracy points and 1.7 AUC points over static IRT, and the full model with structured multidimensional proficiencies gains 2.8 and 2.4. Its closed form is derived for a probit link, so logistic item parameters need conversion. VTIRT [32] provides amortised variational inference for dynamic IRT but assumes one item per time step and was evaluated with thousands of learners; its behaviour with a few long trajectories is untested. GD-GPIRT [33] uses Gaussian processes to let both ability and item response functions drift, at high computational cost, and validates by forecasting several periods ahead. Yan, Tang and Shimada [34] derive a detectability bound for item-parameter drift under a state-space model and find that on four standard KT datasets the observed drift is far below the minimum detectable drift; a first-order application of their equal-bin bound to our data density gives about 2.3 logits per item per checkpoint, which means per-item drift tests must be re-posed at the item-bank level or in coarse time bins, and a project-specific simulation-based power analysis is needed because the bound assumes independent observations. Wang, Berger and Burdick [35] is a foundational Bayesian treatment of dynamic IRT supporting both retrospective and online inference.

LLMs used for knowledge tracing of human students

A large recent literature uses LLMs to trace human students: LLM-KT [36] aligns an LLM with a KT objective, NTKT [37] reframes KT as next-token prediction, CLST [38] addresses cold start, and Scarlatos et al. [39] trace knowledge in tutor-student dialogues. These are the reverse of our direction (the LLM is the tracer, not the learner) and are cited to prevent confusion; our title and introduction state the direction explicitly.

Visualization of training and evaluation

DeepTracker [40] visualised a CNN's per-validation-image correctness across training iterations as a pixel chart with class-level rollups; it is the closest visual precedent for our dashboard. Zeno [41] offers per-slice sparklines across model versions and accepts lm-evaluation-harness output, so it can approximate per-skill curves over checkpoints without a learner model. LLM Comparator [42] shows side-by-side prompt-level comparisons of two models. The PSLC DataShop learning-curve report [43] is the display grammar used in education for per-knowledge-component curves. No existing tool shows a fitted per-skill mastery trajectory with uncertainty across training checkpoints.

1.2 State-of-the-Art Summary

For estimating what an LLM knows, the state of the art is static psychometrics. Fluid Benchmarking [8] fits 2PL IRT to leaderboard results, releases item parameters, and selects items adaptively during pretraining; Item Response Scaling Laws [9] scales the same idea to 6,612 checkpoints and folds ability into a scaling law; ATLAS [17] provides calibrated item banks; and Beyond Scores [13] adds a skill dimension through a Q-matrix. All of these estimate a fixed ability (or profile) for each frozen model or checkpoint independently. None maintains a state that carries information from one checkpoint to the next, none models forgetting, and none is validated by predicting the responses of a later checkpoint.

For describing how skills develop during training, the state of the art is accuracy curves with thresholds. MathCAMPS [21] and the Implicit Curriculum paper [22] show that skills emerge in a stable, curriculum-like order; Chang et al. [23] show power-law forgetting of injected facts; Harmon et al. [24] count item-level forgetting after post-training; Signal and Noise [18] quantifies how noisy a single checkpoint's score is and shows that averaging checkpoints helps. These works observe the phenomena we care about (emergence, forgetting, noise) without a per-skill latent learner state: Liu et al. do forecast held-out task-level trajectories, and Hu et al. [26] fit latent states to aggregate metrics, but none forecasts item-level responses at later checkpoints from a per-skill state with a transition kernel.

For learner modelling itself, the state of the art in education is deep KT (AKT [4], SAINT [45]) trained on large student populations, with BKT variants that include forgetting [2] remaining competitive and far more interpretable. For few learners observed over many periods, the closest precedents are random-walk dynamic IRT as in Martin and Quinn [29] with the emIRT implementation [30], which needs adaptation to share item parameters across periods, and T-SKIRT [31] as a cheap closed-form temporal correction to static IRT. The pyBKT library [6] is the maintained BKT implementation; batching hundreds of observations per step in it requires a log-space E-step.

Our project combines these: it treats each checkpoint as one moment in a learner's history, fits sequential learner models (BKT with forgetting, a Martin-Quinn-style dynamic IRT with tied item parameters, T-SKIRT, and deep KT as an ablation) to item-level checkpoint responses, validates them by forecasting held-out later checkpoints at the item level, tests whether item parameters stay stable across a single model's training run, and presents the fitted trajectories in an interactive tool. In the sources reviewed through September 30, 2026, we did not identify a method that jointly models a stochastic per-skill latent state across an LLM's pretraining checkpoints, treats each checkpoint's responses as a batch, and evaluates item-level

predictions on later held-out checkpoints with calibrated uncertainty. Related work already provides checkpoint psychometrics [8, 9], scaling forecasts [9], task-level emergence prediction [22], aggregate latent training phases [26], and static skill diagnosis [13, 15, 16]. Our study will test whether sequential state modelling adds predictive and diagnostic value beyond these.

On the data side, the state of the art has moved quickly. Fluid Benchmarking's release (six models, 452 checkpoints, 27,842 usable items) is in hand and ingested. DataDecide-eval-instances [44] (about 150,000 checkpoints of models up to 1B parameters and 500 million item-level evaluations according to its dataset card, ODC-BY) and the Implicit Curriculum repository [22] (per-item results for Amber, OLMo-2 and Pythia-6.9B on 91 skill-structured tasks, MIT) are public, and we found no learner model fitted to either. Phase 1 therefore needs no GPU time. The preferred Phase 2 extension (OLMo-2 on MathCAMPS) does need GPU access; if it cannot be secured, these released datasets provide a no-inference fallback with a different skill taxonomy.

1.3 References

[1] Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction, 4(4), 253–278.

The original Bayesian Knowledge Tracing paper. Defines the two-state HMM per skill with learn, guess and slip parameters that our BKT branch uses.

[2] Khajah, M., Lindsey, R. V., & Mozer, M. C. (2016). How deep is knowledge tracing? Proceedings of the 9th International Conference on Educational Data Mining. arXiv:1604.02416.

Shows that BKT extended with forgetting, ability and skill discovery recovers about half of DKT's advantage. Motivates BKT with forgetting as a primary model given our small number of learners.

[3] Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., & Sohl-Dickstein, J. (2015). Deep knowledge tracing. Advances in Neural Information Processing Systems, 28.

Introduces LSTM-based knowledge tracing. Our deep KT ablation follows this family; the paper also establishes the next-response prediction protocol we adopt.

[4] Ghosh, A., Heffernan, N., & Lan, A. S. (2020). Context-aware attentive knowledge tracing. Proceedings of KDD 2020. arXiv:2007.12324.

Attention-based KT with item difficulty embedding. Representative of modern deep KT, developed and evaluated on large student populations; its behaviour with a handful of learners is not established.

[5] Vie, J.-J., & Kashima, H. (2023). Deep knowledge tracing is an implicit dynamic multidimensional item response theory model. Proceedings of ICCE 2023. arXiv:2309.12334.

Theoretical bridge between KT and IRT. Justifies treating BKT and temporal IRT as two members of one model family in our design.

[6] Badrinath, A., Wang, F., & Pardos, Z. (2021). pyBKT: An accessible Python library of Bayesian knowledge tracing models. Proceedings of EDM 2021. arXiv: 2105.00385.

Maintained MIT-licensed BKT implementation with forgetting, per-item guess/slip and multiple learn rates. Its internal data format supports batched observations per time step, which we rely on.

[7] Choffin, B., Popineau, F., Bourda, Y., & Vie, J.-J. (2019). DAS3H: Modeling student learning and forgetting for optimally scheduling distributed practice of skills. Proceedings of EDM 2019. arXiv:1905.06873.

Logistic model over item, skill and time-window features. Adopted as a strong non-latent baseline.

[8] Hofmann, V., Heineman, D., Magnusson, I., Lo, K., Dodge, J., Sap, M., Koh, P. W., Wang, C., Hajishirzi, H., & Smith, N. A. (2025). Fluid language model benchmarking. Proceedings of COLM 2025. arXiv:2509.11106.

Static 2PL IRT plus adaptive item selection over pretraining checkpoints. The paper evaluates 467 checkpoints of six models; the public release we ingested contains 452. Source of our Phase 1 dataset and our static-IRT baseline; also reports that random-subset accuracy is non-monotonic late in training, a competing explanation for forgetting that we must rule out.

[9] Truong, S., Tu, Y., Schaeffer, R., & Koyejo, S. (2026). Item response scaling laws: A measurement theory approach for efficient and generalizable neural scaling estimation. arXiv:2606.07616.

Fits IRT to 6,612 pretraining checkpoints, estimating each checkpoint's ability independently and then fitting a log-compute scaling law through the estimates. The closest psychometric prior work; no sequential state, frozen item parameters, no per-skill decomposition, and validation across sizes and item subsets rather than by forecasting later checkpoints within a run.

[10] Jiang, H., Kwon, S., Luo, J., Xiao, Z., & Zhang, S. (2026). Can we trust item response theory for AI evaluation? arXiv:2607.15190.

Simulation study of IRT estimator reliability. Finds N = 30 respondents insufficient and N = 100 adequate for item recovery in its conditions (sizes between were not tested), that skewed ability distributions harm ranking recovery, and recommends a parametric-

recovery simulation with one's own fitted parameters, which we adopt as a validation step.

[11] Zheng, Q., & Yang, Y. (2026). Are near-tied LLM rankings robust to family-DIF-guided benchmark recomposition? arXiv:2609.00482.

Differential item functioning across model families on MMLU, HellaSwag and WinoGrande. Establishes item non-invariance across populations; our test is within one model across its checkpoints.

[12] Paquin, D., & Jain, R. (2026). What does MMLU actually measure? A psychometric audit of difficulty structure in aggregate benchmark scores. arXiv: 2609.09372.

2PL audit of all 14,042 MMLU items over 1,000 models. Reports a 0.965 latent correlation between STEM and non-STEM ability, concludes the two are distinguishable (a two-dimensional model fits held-out responses better), and recommends clustering standard errors by subject; both shape our MMLU-subject analysis.

[13] Zhang, X., Gong, X., Qin, J., Wang, Q., Liao, J., Wang, Z., Feng, D., & Ding, B. (2026). Beyond scores: Diagnostic LLM evaluation via fine-grained abilities. arXiv:2604.12191.

Multidimensional IRT with a 35-dimension mathematics Q-matrix over 41 frozen models, validated by held-out item prediction. Static per-skill profiling; we cite it to position per-skill trajectories as the new part. Its AUC is not comparable to ours because its held-out items use default item parameters.

[14] Mandujano Reyes, J. F. (2026). Laplace-PSN-IRT: Uncertainty quantification for neural item response theory models of LLM benchmarks. arXiv:2607.25257.

Approximately calibrated posterior over LLM ability via a Laplace approximation; the item branch is poorly calibrated when discrimination and difficulty are sampled jointly. Bounds what we may claim about uncertainty.

[15] Kuang, Z., Zhu, F., Jiang, M., Lai, Y., Wang, Z., Wang, Z., Qiu, M., Huang, J., Peng, M., Xie, Q., & Ananiadou, S. (2025). From scores to skills: A cognitive diagnosis framework for evaluating financial large language models. arXiv: 2508.13491.

Cognitive diagnosis of 30 frozen financial LLMs at the skill level. Static analogue of our per-skill profile.

[16] Zhao, H., Han, J., Liang, J., Xiao, Y., Meng, X., & Wei, J. (2025). CDS: Knowledge component-driven data synthesis guided by cognitive diagnosis theory. arXiv:2501.07674.

Uses per-skill cognitive diagnosis of an LLM to synthesise training data for weak skills. The downstream application our mastery estimates would feed.

[17] Li, P., et al. (2025). Adaptive testing for LLM evaluation: A psychometric alternative to static benchmarks. arXiv:2511.04689 (v3, ICML 2026 spotlight).

Releases calibrated item banks (primarily 3PL, with a 2PL comparison) over 3,000 models for five benchmarks; MMLU is not among them. Used to cross-check Fluid Benchmarking's calibration where items overlap.

[18] Heineman, D., Hofmann, V., Magnusson, I., Gu, Y., Smith, N. A., Hajishirzi, H., Lo, K., & Dodge, J. (2025). Signal and noise: A framework for reducing uncertainty in language model evaluation. Advances in Neural Information Processing Systems, 38. arXiv:2508.13144.

Defines checkpoint-to-checkpoint noise, shows averaging checkpoints and filtering low-SNR MMLU subtasks improve decision accuracy. The aggregate-level precedent for our smoothing baseline.

[19] Biderman, S., et al. (2023). Pythia: A suite for analyzing large language models across training and scaling. Proceedings of ICML 2023. arXiv:2304.01373.

Sixteen models (at eight parameter sizes, with deduplicated-data variants) with 154 public checkpoints each and identical data order. Two Pythia models are in our Phase 1 data; the full suite is a Phase 2 option.

[20] Groeneveld, D., et al. (2024). OLMo: Accelerating the science of language models. Proceedings of ACL 2024. arXiv:2402.00838.

Fully open model family with hundreds of released checkpoints. OLMo-1 7B is in our Phase 1 data. See [47] for OLMo 2.

[21] Mishra, S., Poesia, G., & Goodman, N. D. (2025). From next-token to mathematics: The learning dynamics of mathematical reasoning in language models. Proceedings of COLM 2025. arXiv:2407.00900.

MathCAMPS: 44 Common Core standards, skills emerge in curriculum order across Pythia, OLMo-2 and Amber checkpoints; also analyses post-training effects and robustness to follow-up questions. Its pretraining analysis reports per-skill accuracy curves without a sequential learner model. Our Phase 2 dataset and the per-skill accuracy-curve baseline we must beat.

[22] Liu, E., Sun, K., Li, M., Lee, I., Tjuatja, L., Huang, J., & Neubig, G. (2026). What do language models learn and when? The implicit curriculum hypothesis. arXiv: 2604.08510.

Threshold-based emergence order across 91 tasks and nine models; composite skills usually follow their components; predicts held-out composite-task trajectories from function vectors. Its repository releases per-item results for three of our learners on a skill-structured suite.

[23] Chang, H., Park, J., Ye, S., Yang, S., Seo, Y., Chang, D.-S., & Seo, M. (2024). How do large language models acquire factual knowledge during pretraining? Advances in Neural Information Processing Systems, 37. arXiv:2406.11813.

Injects fictional facts into OLMo pretraining and finds power-law forgetting of the injected knowledge. Motivates testing non-constant or time-scaled forgetting in our models; a different observable from ours, so not a direct prior.

[24] Harmon, J., Hochlehnert, A., Bethge, M., & Prabhu, A. (2025). Mapping post-training forgetting in language models at scale. arXiv:2510.17776.

Item-level correct-to-incorrect transition counts across about 100 subdomains after post-training. The counting baseline for forgetting.

[25] Liao, C., Xie, R., Sun, X., Sun, H., & Kang, Z. (2025). Exploring forgetting in large language model pre-training. Proceedings of ACL 2025. arXiv:2410.17018.

Entity-level forgetting during pretraining with new retention metrics. Related evidence that forgetting during pretraining is real.

[26] Hu, M. Y., Chen, A., Saphra, N., & Cho, K. (2023). Latent state models of training dynamics. Transactions on Machine Learning Research. arXiv:2308.09543.

Fits HMMs to aggregate training metrics to find latent phases. Methodological precedent for a latent-state view of training, at the aggregate rather than skill level.

[27] Michaelov, J. A., Levy, R. P., & Bergen, B. K. (2025). Language model behavioral phases are consistent across architecture, training data, and scale. Advances in Neural Information Processing Systems, 38. arXiv:2510.24963.

Shows structured behavioural phases across 1,400 checkpoints. Motivation that pretraining has phases worth modelling.

[28] Dominguez-Olmedo, R., Dorner, F. E., & Hardt, M. (2025). Training on the test task confounds evaluation and emergence. Proceedings of ICLR 2025. arXiv: 2407.07890.

Shows Pythia sits at chance on MMLU and that family differences shrink after equal fine-tuning on task data. A competing explanation for our cross-model tiers that we address in the discussion.

[29] Martin, A. D., & Quinn, K. M. (2002). Dynamic ideal point estimation via Markov chain Monte Carlo for the U.S. Supreme Court, 1953–1999. Political Analysis, 10(2), 134–153.

Dynamic ideal-point model for 29 Supreme Court justices over 47 terms with a random-walk prior on ability, estimated by MCMC with Kalman forward-filtering and backward-sampling; cases differ across terms. Structural precedent for our primary temporal-IRT model, which additionally needs item parameters tied across periods.

[30] Imai, K., Lo, J., & Olmsted, J. (2016). Fast estimation of ideal points with massive data. American Political Science Review, 110(4), 631–656.

Variational EM implementation of the Martin-Quinn model in the R package emIRT (function dynIRT, version 0.0.15, GPL-3). Each item is assigned to one period and the manual warns that the variational variances are unusable as uncertainty; we use it as a reference implementation and starting point, not a drop-in.

[31] Ekanadham, C., & Karklin, Y. (2017). T-SKIRT: Online estimation of student proficiency in an adaptive learning system. arXiv:1702.04282.

Temporal IRT with Wiener-process ability and closed-form time-decayed discrimination under a probit link; temporal-only ablation gains +2.2 accuracy / +1.7 AUC over static IRT, full model +2.8 / +2.4. Cheap first temporal model to run on our data, after converting Fluid's logistic item parameters.

[32] Kim, Y., Sankaranarayanan, S., Piech, C., & Thille, C. (2023). Variational temporal IRT: Fast, accurate, and explainable inference of dynamic learner proficiency. Proceedings of EDM 2023. arXiv:2311.08594.

Amortised variational dynamic IRT. Assumes one item per time step and was evaluated with thousands of learners; its recognition network is trained across learners, so its behaviour with six long trajectories is untested. Not adopted; cited as the fast-inference state of the art.

[33] Chen, Y., Montgomery, J. M., & Garnett, R. (2025). A dynamic, ordinal Gaussian process item response theoretic model. arXiv:2504.02643.

Gaussian-process dynamic IRT with time-varying item parameters and a forecast-ahead validation protocol. Used on subsamples for the item-drift question.

[34] Yan, X., Tang, C., & Shimada, A. (2026). Recovering stranded discrimination in knowledge tracing: Per-item bias correction via empirical-Bayes shrinkage. arXiv:2606.14123.

State-space model of item-parameter drift with a detectability bound. Shows observed drift is below the detectable minimum on standard KT datasets; we use its bound to design our drift test.

[35] Wang, X., Berger, J. O., & Burdick, D. S. (2013). Bayesian analysis of dynamic item response models in educational testing. Annals of Applied Statistics, 7(1), 126–153.

Canonical Bayesian dynamic IRT reference. Background for the full-posterior refit of our final models.

[36] Wang, Z., et al. (2025). LLM-KT: Aligning large language models with knowledge tracing using a plug-and-play instruction. arXiv:2502.02945.

An LLM used to trace human students. Reverse direction from ours; cited to prevent confusion.

[37] Norris, M., Gal, K., & Bulathwela, S. (2025). Next token knowledge tracing: Exploiting pretrained LLM representations to decode student behaviour. arXiv: 2511.02599.

Reframes KT as next-token prediction over student histories. Reverse direction from ours.

[38] Jung, H., et al. (2024). CLST: Cold-start mitigation in knowledge tracing by aligning a generative language model as a students' knowledge tracer. arXiv: 2406.10296.

LLM as a tracer of human students under cold start. Reverse direction from ours.

[39] Scarlatos, A., Baker, R. S., & Lan, A. (2025). Exploring knowledge tracing in tutor-student dialogues using LLMs. Proceedings of LAK 2025. arXiv: 2409.16490.

Applies KT to dialogue turns with LLM-identified knowledge components. Reverse direction; also a precedent for KT on unconventional time steps.

[40] Liu, D., Cui, W., Jin, K., Guo, Y., & Qu, H. (2018). DeepTracker: Visualizing the training process of convolutional neural networks. ACM Transactions on Intelligent Systems and Technology, 10(1). arXiv:1808.08531.

Per-instance correctness across training iterations as a pixel chart with class rollups. The closest visual precedent for our dashboard.

[41] Cabrera, Á. A., Fu, E., Bertucci, D., Holstein, K., Talwalkar, A., Hong, J. I., & Perer, A. (2023). Zeno: An interactive framework for behavioral evaluation of machine learning. Proceedings of CHI 2023. arXiv:2302.04732.

Slice-based evaluation with per-slice sparklines across model versions. The practical substitute a reviewer would name; we position our tool against it.

[42] Kahng, M., Tenney, I., Pushkarna, M., Liu, M. X., Wexler, J., Reif, E., Kallarackal, K., Chang, M., Terry, M., & Dixon, L. (2024). LLM Comparator: Visual analytics for side-by-side evaluation of large language models. CHI 2024 Extended Abstracts. arXiv:2402.10524.

Side-by-side model comparison by prompt category. Shows skill-grouped views exist; skill-grouped views over training time do not.

[43] Koedinger, K. R., Baker, R. S., Cunningham, K., Skogsholm, A., Leber, B., & Stamper, J. (2010). A data repository for the EDM community: The PSLC DataShop. In Handbook of Educational Data Mining (pp. 43–56). CRC Press.

Learning-curve reports per knowledge component. The display grammar we transfer to LLM checkpoints.

[44] Magnusson, I., et al. (2025). DataDecide: How to predict best pretraining data with small experiments. arXiv:2504.11393.

Its dataset card describes about 150,000 checkpoints of models up to 1B parameters with 500 million item-level evaluations (ODC-BY); we have verified the card and file layout, not recounted the totals. A no-GPU Phase 2 data option.

[45] Choi, Y., Lee, Y., Cho, J., Baek, J., Kim, B., Cha, Y., Shin, D., Bae, C., & Heo, J. (2020). Towards an appropriate query, key, and value computation for knowledge tracing. Proceedings of L@S 2020. arXiv:2002.07033.

SAINT: transformer-based knowledge tracing. Representative of deep KT that needs large student populations.

[46] Chen, M., Roberts, N., Bhatia, K., Wang, J., Zhang, C., Sala, F., & Ré, C. (2023). Skill-it! A data-driven skills framework for understanding and training language models. Advances in Neural Information Processing Systems, 36. arXiv: 2307.14430.

Online data selection that orders training data by skill prerequisites. Together with CDS [16], the kind of training-time use a per-skill mastery estimate would feed.

[47] OLMo Team. (2025). 2 OLMo 2 Furious. arXiv:2501.00656.

OLMo 2 technical report. OLMo-2 7B is in our Phase 1 data and is the Phase 2 target; its released intermediate checkpoints and training artifacts are what "public training history" refers to.

[48] Kübler, J., Budhathoki, K., Kleindessner, M., Zhou, X., Yin, J., Khetan, A., & Karypis, G. (2026). When LLMs get significantly worse: A statistical approach to detect model degradations. Proceedings of ICLR 2026. arXiv:2602.10144.

Exact one-sided McNemar test on paired item responses with pooled, max-drop and Fisher aggregation across tasks. The paired test we will use for forgetting; selection of the peak across checkpoints and screening across subjects must be built into the test family or a null simulation.

[49] Yeung, C.-K., & Yeung, D.-Y. (2018). Addressing two problems in deep knowledge tracing via prediction-consistent regularization. Proceedings of L@S 2018. arXiv:1806.02180.

Identifies reconstruction and inconsistency problems in DKT's predictions. Cited for the known weaknesses of deep KT outputs as mastery estimates.

Chapter 2
Project Justification

Large language models are evaluated with a single benchmark score per checkpoint. That number cannot say which skills a model has acquired, when each one appeared, whether any were lost later in training, or how confident anyone should be in the estimate. Developers who run pretraining, and researchers who study it, currently answer those questions by eye from accuracy curves. Intelligent tutoring systems addressed the same problem for human students thirty years ago with knowledge tracing, which maintains a per-skill mastery estimate, updates it after each response, and is validated by predicting the learner's next response. The literature search in Chapter 1 (two passes, September 22 and 30, 2026, the second spot-verified against primary sources and then independently reviewed) found that the closest prior work either fits a static ability to each checkpoint independently (Item Response Scaling Laws [9], Fluid Benchmarking [8]), describes emergence and forgetting with accuracy thresholds and transition counts (MathCAMPS [21], Implicit Curriculum [22], Harmon et al. [24]), or models latent phases of training at the aggregate level (Hu et al. [26]). In the sources reviewed, we did not identify a method that maintains a stochastic per-skill state with a transition kernel over checkpoints, treats each checkpoint's responses as a batch, forecasts item-level responses at later held-out checkpoints with calibrated uncertainty, and tests whether item parameters stay stable across a single model's run. A search cannot prove that no such method exists anywhere; it establishes that the question is open in the literature we can find.

The project is feasible within two semesters because the hardest dependency is already resolved. Fluid Benchmarking's release provides per-item, per-checkpoint correctness for six open models across 452 checkpoints and 27,842 items, already ingested into a canonical table of 12.6 million binary responses, so Phase 1 needs no GPU. The preferred Phase 2 extension (OLMo-2 [47] on MathCAMPS) needs GPU access; DataDecide [44] and the Implicit Curriculum repository [22] provide a no-inference fallback. The planned models have open-source starting points or fit in under a hundred lines, and each needs an adaptation that is identified and scheduled in Chapters 3, 4 and 9: a log-space E-step for batched BKT, tied item parameters and a separate uncertainty procedure for the dynamic IRT, and conversion of Fluid's logistic item parameters to T-SKIRT's probit scale. The evaluation harness, five trivial baselines and first per-skill learning curves exist and run in minutes.

The project aims at a peer-reviewed publication; our advisor identified NeurIPS as the target. The research question is open in the reviewed literature, the baselines are cited NeurIPS and COLM methods, and the success criterion is pre-committed: beat

the strongest trivial predictor on the same benchmark subset and horizon (pooled, log-loss 0.360 and within-skill AUC 0.80 one checkpoint ahead; MMLU alone, 0.483 and 0.79). Whether the fitted state measures transferable mastery, and whether mastery-guided training helps, are questions to test. Three risks identified in the review are treated as planned analyses (details in Chapter 4): how separable the 57 MMLU subject trajectories are within a run; whether item-parameter drift is detectable at our data density, to be settled by a simulation-based power analysis rather than assumed; and how fast the trivial baselines decay with forecast horizon. The practical value is direct: a per-skill mastery estimate with uncertainty tells a developer whether a checkpoint has learned a skill or is guessing, whether a later one has forgotten it, and which skills to target with the next training data, which is the use CDS [16] and Skill-it [46] make of skill diagnoses; the visualization requested by our advisor makes that usable without reading the paper. Semester one delivers the pipeline, baselines and first validated learner model on public data; semester two delivers the Phase 2 extension, the drift and forgetting analyses, the visualization, and the paper.

Chapter 3
Identify Baseline Approaches

We group baselines into three tiers: trivial predictors that any learner model must beat on next-checkpoint prediction; published methods that describe the same phenomena without a learner model; and the learner-model family itself, where the question is which member fits our data. For each we identify the state-of-the-art method, the implementation we will use, and its status.

3.1 Tier 1: Trivial next-checkpoint predictors (implemented, results in hand)

These have no latent state. They were implemented in our evaluation harness (src/llm_kt/baselines.py) and scored on all six models of the Phase 1 data using a strict walk-forward protocol: at each origin checkpoint t the predictor sees every response at checkpoints at most t and predicts the probability of a correct answer for every item at checkpoint t+h. Results pooled over six models:

Predictor	Description	h=1 log-loss	h=1 AUC	h=5 log-loss	h=5 AUC
global_mean	Overall accuracy at the last checkpoint	0.661	0.50	0.664	0.50
skill_mean	Per-skill accuracy at the last checkpoint	0.582	0.50	0.585	0.50
copy_last	Item's last correctness, softened (eps=0.1)	0.432	0.73	0.473	0.70
copy_last_calibrated	Two-state Markov chain on observed correctness, per skill	0.378	0.73	0.404	0.70
item_mean	Item's history average shrunk toward the global mean (inspired by checkpoint averaging [18])	0.360	0.80	0.388	0.78

How to read the table. Log-loss and Brier are weighted by item count over (model, target checkpoint, skill) slices, so models with more checkpoints weigh more; AUC is the unweighted mean over slices where both classes occur. AUC is computed within a skill slice, so any predictor that is constant within a skill (global_mean, skill_mean, and a BKT with one guess/slip pair per skill) scores 0.50 by construction. The pooled numbers cover all five retained benchmarks; on MMLU alone item_mean scores log-loss 0.483 and AUC 0.79 at h=1, and a learner model fitted on MMLU must

be compared against that. copy_last_calibrated applies its one-step transition probabilities at every horizon, so its h=5 score is that of a fixed predictor, not a five-step Markov forecast. The pre-committed bar for any learner model is therefore: beat item_mean on log-loss and within-skill AUC on the same benchmark subset and horizon. The literature review recommends adding DAS3H [7] (open source, github.com/BenoitChoffin/das3h) as a sixth, learned, non-latent baseline because it combines item difficulty with recent-history features, which is most of the signal in our data; its time-window features must be redefined over training progress.

3.2 Tier 2: Published descriptive methods (open source, to be reproduced)

- Static IRT per checkpoint: Fluid Benchmarking [8]. Code and data at github.com/allenai/fluid-benchmarking; the release includes fitted 2PL item parameters (a, b) for every item, so a static-IRT ability per checkpoint per skill can be computed directly. This is the psychometric baseline and the item-parameter initialisation for our temporal models. Two uses must be kept separate: as a forecast, the ability is estimated from history up to the origin only and carried forward (static persistence); as a diagnosis, the ability is estimated on part of a checkpoint's items and scored on the held-out items of the same checkpoint. An ability fitted on a checkpoint's answers is never scored on those same answers. Status: data ingested; ability fitting is the next task.
- Per-skill accuracy curves: MathCAMPS [21]. Code at github.com/gpoesia/mathcamps. Reproduced on our data as raw accuracy per (model, skill, checkpoint) with a 95% Wilson interval (src/llm_kt/curves.py). Status: implemented; figures generated for OLMo-2 across MMLU subjects and for six models on one subject.
- Threshold-based emergence: Implicit Curriculum [22]. Code and per-item results at github.com/KaiserWhoLearns/ElementalTask (MIT). Emergence is the first checkpoint at which accuracy exceeds a threshold. Status: trivial to compute from our curves table; will be reported alongside model-based emergence.
- Item-level transition counting for forgetting: Harmon et al. [24]. No learner model; forgetting is the count of items correct at t and incorrect at t+h. Status: will be computed per subject, with the exact one-sided McNemar test and max-drop aggregation from Kübler et al. [48], to decide whether our seven candidate subjects (peak-minus-final accuracy above 0.08) show a statistically real paired drop. Because the peak was chosen after seeing each curve and 57 subjects were screened, the test family or a null simulation must include that selection; a significant paired drop is behavioural evidence, not proof of latent forgetting.

- Static skill profiles: Beyond Scores [13] (multidimensional IRT with a Q-matrix) and cognitive diagnosis [15]. These profile a frozen model; we compare our final-checkpoint profile against a static fit on the same checkpoint to show what the temporal model adds. Status: planned for Workbook 2.

3.3 Tier 3: Learner-model family (the methods under study)

Model	Implementation	Licence / status	Role
BKT with forgetting	pyBKT [6], github.com/CAHLR/pyBKT (pin a commit), internal Data format: one column per checkpoint with many observations and one transition per gap	MIT. Its E-step multiplies raw emissions before normalising and underflows for batches of hundreds of items (found by reading EM_fit.py, not by an executed test); needs a log-space E-step, either patched or reimplemented in NumPy.	Interpretable model: per-skill initial mastery, learn, forget, guess, slip. With one guess/slip pair per skill its within-skill AUC is 0.50 by construction, so it can compete on log-loss but not on AUC; a per-item guess/slip variant (multigs) is required to compete on AUC and needs a parameter-tying or shrinkage plan to remain estimable with six learners.
Dynamic IRT (Martin-Quinn style)	Reference implementation emIRT::dynIRT [30], CRAN 0.0.15; custom NumPy Kalman filter/smoother for our variant	GPL-3. dynIRT assigns each item to one period, so tying each item's (a, b) across all checkpoints requires a model change, not a patch; uneven gaps need a gap-scaled transition variance; uncertainty must come from a parametric bootstrap or MCMC refit because its variational variances are documented as unusable. Feasibility risk recorded in Chapter 4.	Primary temporal-IRT model: continuous per-skill ability with item parameters shared across checkpoints, elapsed-training transition, and identification constraints (anchor items or fixed item parameters from Fluid's calibration).
T-SKIRT	No author release; ~50 lines from the paper [31]	Reimplement. Probit link: convert Fluid's logistic 2PL parameters to the probit scale (discrimination divided by about 1.7, difficulty unchanged) or refit under a probit model.	First temporal model to run; answers whether temporality helps over static IRT.

Model	Implementation	Licence / status	Role
GD-GPIRT	github.com/yahoochen97/OrdGPIRT [33]	No licence; R/MATLAB; expensive; runnable entry point for the GD variant not yet verified.	Subsample only, for the item-drift question.
Deep KT (DKT, AKT)	pyKT, github.com/pykt-team/pykt-toolkit	MIT.	Ablation only, on Phase 1 data (six models), with (model, subject) pseudo-learners and a leave-one-model-out split; on Phase 2 only if more than one model is evaluated there.
VTIRT	github.com/yunsungkim0908/vtirt [32]	No licence file; one item per step.	Not adopted; cited.

Four validation steps apply to every Tier 3 model. (1) A parametric-recovery simulation using our own fitted parameters as the generating model over six autocorrelated trajectories, as recommended by Jiang et al. [10], reporting recovery at our actual design. (2) A unit test that the number of state transitions equals the number of checkpoint gaps (T minus 1 for T checkpoints, with an initial state at the first) and that no transition occurs between items at the same checkpoint. (3) Chronological validation and test blocks fixed before model selection, with hyperparameters chosen on the validation block only, so that the exploratory baseline results reported here do not leak into model choice. (4) Interval coverage checks for any reported uncertainty: Wilson bands describe raw accuracy, not posterior mastery, and clustering by subject does not by itself address dependence across checkpoints or models.

Chapter 4
Dependencies and Deliverables

4.1 Dependencies

Only showstoppers are listed.

- Per-item, per-checkpoint correctness data for Phase 1. Resolved on September 29, 2026: allenai/fluid-benchmarking on HuggingFace contains it for six models (Amber-6.7B, K2-65B, OLMo-1 7B, OLMo-2 7B, Pythia-2.8B, Pythia-6.9B), 452 checkpoints, 27,842 usable items. Ingested into data/processed/fluid_responses.parquet. Fallback if the release were withdrawn: DataDecide-eval-instances (ODC-BY) and the ElementalTask repository (MIT), both verified public.
- GPU access for the preferred Phase 2 (OLMo-2 checkpoints on MathCAMPS). Not yet secured. Roughly 50 to 100 checkpoints of a 7B model on a planned subset of MathCAMPS problems (count to be fixed from the dataset manifest). Options: SJSU HPC cluster, Google Colab Pro, or rented cloud GPUs (Lambda, RunPod). Owner: Aniket. A pilot throughput and cost estimate is due 2026-11-25; the go/no-go decision is due 2026-12-12. If no GPU is available, Phase 2 uses the no-inference fallback datasets above, which lose the Common Core skill taxonomy but keep a skill-decomposed structure; the fallback requires item-ID and coverage validation before it is adopted.
- Skill tags. Phase 1 uses MMLU's 57 subjects, which exist in the item ids. Risk: the MMLU psychometric audit [12] finds STEM and non-STEM abilities correlate at 0.965 across final models (while still being distinguishable), so subject trajectories may move largely together. Mitigation: measure inter-subject trajectory correlation as soon as the first model is fit; fall back to a four-category rollup as the primary unit if needed.
- The dynamic IRT model itself. No off-the-shelf implementation fits our design (item parameters shared across checkpoints, batched observations per checkpoint, uneven gaps, usable uncertainty). emIRT::dynIRT is a reference point, not a drop-in; the plan is a NumPy Kalman filter/smoother with tied item parameters (about 200 to 400 lines) validated by the recovery simulation. If that model cannot be made identifiable and stable by the end of Sprint 5, T-SKIRT with static item parameters becomes the primary temporal model and the dynamic IRT is reported as an attempted extension.

- Batched BKT. pyBKT's E-step underflows for our batch sizes; a log-space forward-backward is required. This is a bounded engineering task (Sprint 4) with a fallback to a NumPy reimplementation of the same HMM.
- Time metadata. Step is missing for Amber and K2, and tokens seen is missing for Amber, K2 and Pythia in the Fluid release. Pythia's can be derived (2,097,152 tokens per step); Amber's and K2's come from LLM360's published schedules. Without them, temporal models must use checkpoint index as the clock, which is what the current baselines do.
- Team availability. Three members with coursework; if one member is unavailable for more than two weeks, the visualization track is the one to defer, since the paper does not depend on it.

4.2 Deliverables

- Canonical dataset and ingest pipeline: a long-format Parquet table of (model, checkpoint, item, skill, correct) built reproducibly from public releases, with a tested ingest script (delivered for Phase 1; Phase 2 extends it).
- Learner-model library: Python package llm_kt with BKT-with-forgetting, T-SKIRT, dynamic IRT wrappers, and deep KT ablation, all exposing one Predictor interface.
- Evaluation harness: walk-forward next-checkpoint prediction at multiple horizons with log-loss, Brier, accuracy and AUC, per (model, checkpoint, skill), plus subject-clustered standard errors and the parametric-recovery simulation (delivered in initial form).
- Fitted mastery trajectories: a released dataset of per-skill mastery estimates with uncertainty for every checkpoint of every model studied, plus item-parameter drift estimates.
- Interactive visualization: a web application (FastAPI backend serving fitted trajectories, React frontend) showing per-skill mastery over checkpoints with uncertainty, emergence and forgetting events, item-difficulty drift, and cross-model comparison. Requested by the advisor.
- Analyses: emergence order versus curriculum order, forgetting events that survive the McNemar test, and item-parameter invariance at bank level with the detectability bound reported.
- Publications: a NeurIPS submission (May 2027) and an arXiv preprint; a public GitHub repository with code, data-building scripts and documentation.

Chapter 5
Project Architecture

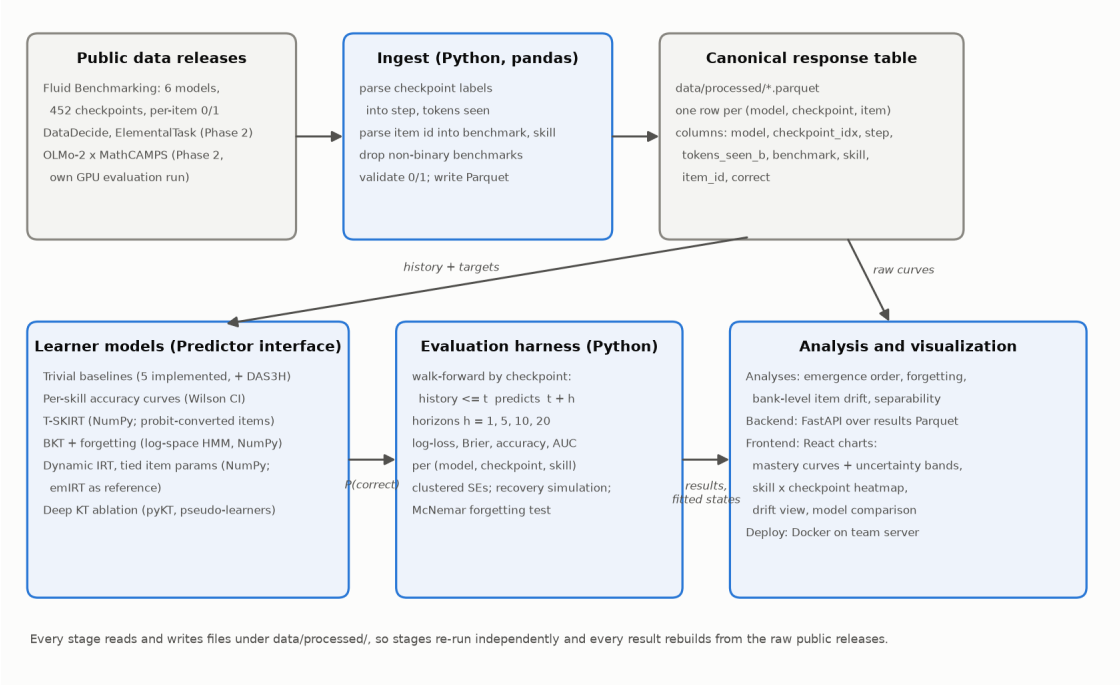


Figure 5.1. System architecture. Left to right: public data releases are ingested into one canonical response table; learner models and baselines read it through a common Predictor interface; the walk-forward harness scores them and writes results; the visualization serves fitted trajectories and results to a browser.

The system is a batch pipeline followed by a serving layer. Each stage reads and writes files in data/processed/, so stages can be re-run independently and results are reproducible from the raw releases.

5.1 Data layer

Function: turn heterogeneous public releases into one table. Sources: Fluid Benchmarking (six per-model CSVs, wide format, checkpoint columns named differently per model family), and in Phase 2 our own OLMo-2 evaluation on MathCAMPS plus optional DataDecide and ElementalTask files. The ingest module (src/llm_kt/ingest.py) parses checkpoint labels into step and tokens-seen, parses item ids into benchmark and skill, drops the non-binary TruthfulQA benchmark, validates that every kept cell is 0 or 1, and writes a long-format Parquet table with columns model, checkpoint_idx, checkpoint_label, step, tokens_seen_b, benchmark, skill, item_id, correct. Technology: Python 3.12, pandas, pyarrow, huggingface_hub. Current size: 12,584,584 rows, 27 MB.

5.2 Model layer

Function: estimate a per-skill knowledge state per checkpoint and predict future responses. Every model implements one interface, Predictor.predict(history, target, horizon) -> probabilities, where history is all responses at checkpoints up to the origin (with step and tokens-seen columns where available), target lists the items to predict, and horizon is the number of checkpoints ahead. Learner models additionally export a fitted-state artifact per (model, skill, checkpoint): filtered and smoothed mastery estimates, their uncertainty, and item-parameter estimates, written to data/processed/ for the analysis and visualization layers. Members: five trivial baselines (implemented), per-skill accuracy curves with Wilson intervals (implemented), T-SKIRT (planned, NumPy), BKT with forgetting (planned, log-space NumPy HMM or patched pyBKT), Martin-Quinn-style dynamic IRT with tied item parameters (planned, NumPy Kalman filter/smoother; emIRT as reference), and deep KT (planned, pyKT, ablation). Technology: Python, NumPy, pandas; R/emIRT for reference fits; PyTorch for the ablation only.

5.3 Evaluation layer

Function: score every model the same way. The harness (src/llm_kt/evaluate.py) walks forward through each model's checkpoints, calls the predictor with history up to origin t and the horizon h, rejects non-finite or out-of-range probabilities, scores predictions at t+h with log-loss, Brier, accuracy and rank-based AUC per (model, target checkpoint, skill), and writes one results Parquet per horizon. Metrics are implemented without external dependencies (src/llm_kt/metrics.py). Planned additions: subject-clustered standard errors and a subject-level block bootstrap, horizons 10 and 20, fixed chronological validation/test blocks, the parametric-recovery simulation, calibration and coverage checks for fitted states, and the McNemar paired test for forgetting. Technology: Python, NumPy, pandas, pytest (50 tests).

5.4 Analysis layer

Function: turn fitted states into the paper's findings. Emergence order per model (model-based and threshold-based), forgetting events that pass the paired test, item-parameter drift at bank level with the detectability bound, inter-subject trajectory correlation, and cross-model comparison. Technology: Python, matplotlib for static figures.

5.5 Visualization layer

Function: let a developer or researcher browse per-skill mastery over training. Backend: FastAPI serving the fitted-trajectory and results Parquet files as JSON

endpoints (per model, per skill, per checkpoint; emergence and forgetting events; item drift). Frontend: React with a charting library, views for mastery-over-checkpoints with uncertainty bands, a skill-by-checkpoint heatmap in the DeepTracker idiom [40], forgetting-event markers, item-difficulty drift, and side-by-side model comparison. Deployment: Docker container on the team's server; static export for the paper's supplementary material. Technology: Python/FastAPI, React, Docker.

5.6 Repository layout

- src/llm_kt/: ingest.py, curves.py, metrics.py, evaluate.py, baselines.py (models to follow).
- scripts/: ingest_fluid.py, plot_skill_curves.py, run_baselines.py.
- tests/: 50 pytest tests covering parsing, aggregation, metrics, harness protocol and baselines.
- data/raw/, data/processed/: gitignored; rebuilt by scripts.
- docs/: project notes, abstract, research brief, literature review, figures, workbook and its review.
- meetings/: one note per advisor or team meeting.

Chapter 6
Evaluation Methodology

To be completed in Workbook Assignment 2. The evaluation protocol already implemented is summarised in Chapters 3 and 5: walk-forward next-checkpoint prediction at horizons 1 and 5 on six models, scored with log-loss, Brier, accuracy and AUC per (model, checkpoint, skill), against five trivial baselines. Workbook 2 will add subject-clustered standard errors, additional horizons, the parametric-recovery simulation, the McNemar forgetting test, and the bank-level item-drift test with its detectability bound.

Chapter 7
System Design/Methodology

7.1 Solution Overview

To be completed in Workbook Assignment 2. The design decisions already made are: the time step is the training checkpoint, not the benchmark item, because the model does not learn from test items; all items at a checkpoint are simultaneous noisy observations of the skill state; primary models are BKT with forgetting and Kalman-filter dynamic IRT, with T-SKIRT as the first temporal model and deep KT as an ablation; Phase 1 skills are the 57 MMLU subjects with a four-category rollup; TruthfulQA is dropped because its scores are continuous.

Chapter 8
Implementation Plan and Progress

8.1 Programming and execution environment (done)

- Python 3.12 virtual environment at the repository root; package llm-kt installed in editable mode from pyproject.toml with dependencies huggingface_hub, pandas, pyarrow, numpy, matplotlib; dev dependencies pytest and ruff.
- Repository layout established (src/, scripts/, tests/, data/, docs/) with data directories gitignored so raw and derived data are always rebuilt from public sources.
- Continuous checks: pytest (50 tests passing) and ruff (clean) run locally before any change is shared. GitHub Actions CI to run both on every push is planned for the next sprint.

8.2 Development tools (per member)

Status as of September 30, 2026. "Verified" means the tool has been installed and used on that member's machine to produce an artifact present in the local repository checkout (the first commit of this work is pending); "to set up" means it is planned with a date and is not yet acquired.

- Ronak (data/infrastructure, harness, visualization). Verified: Git and GitHub, VS Code, Python 3.12 virtual environment with the project package, pytest, ruff, HuggingFace hub client, pandas/pyarrow, matplotlib, Node.js 22, WeasyPrint (all used to build the ingest, harness, figures and this workbook). To set up: FastAPI and React scaffolding (Sprint 7), Docker deployment on the team server (Sprint 12).
- Bhagyesh (learner models). To set up by 2026-10-07: clone the repository, create the virtual environment, run the test suite; R 4.x with emIRT and MCMCpack from CRAN for reference fits; pyBKT pinned to a commit; NumPy/SciPy for the custom Kalman filter. pyKT and PyTorch for the ablation in Sprint 9.
- Aniket (data analyses, statistical tests, Phase 2 evaluation). To set up by 2026-10-07: clone the repository, create the virtual environment, run the test suite; SciPy/statsmodels for the McNemar and bootstrap procedures. lm-evaluation-harness and a GPU environment for the Phase 2 OLMo-2 runs once access is secured (decision by 2026-12-12).

8.3 Hardware, prototyping and simulation environment

- Phase 1 needs no GPU. All ingest, baselines and figures run on a laptop: the full six-model ingest takes 47 seconds; the five-baseline evaluation over 452 checkpoints takes about eight minutes per horizon; figures take seconds.
- Phase 2 (OLMo-2 on MathCAMPS) needs GPU time for roughly 50 to 100 checkpoints of a 7B model. Options under evaluation: SJSU HPC, Colab Pro, rented cloud GPUs. A quote will be obtained before the semester 1 report. The no-GPU fallback datasets (DataDecide, ElementalTask) are verified public.
- The team's self-hosted server (Docker, GitHub Actions to ghcr.io) will host the visualization.

8.4 Sample programs executed and understood (done)

- Data ingest: scripts/ingest_fluid.py downloads the six Fluid Benchmarking CSVs and writes the canonical table. Verified: 12,584,584 rows = 452 checkpoints x 27,842 items, no missing correctness values, correct strictly in {0, 1}, all 57 MMLU subjects present, accuracy rises from first to last checkpoint for every model (for example OLMo-2 7B 0.24 to 0.66). Time metadata is incomplete: step is missing for Amber and K2 (3.3 million cells) and tokens seen for Amber, K2 and Pythia (7.7 million cells); checkpoint index is complete and is the clock used so far.
- Per-skill learning curves: scripts/plot_skill_curves.py produces raw accuracy with Wilson intervals per (model, skill, checkpoint). Findings on OLMo-2 7B (values corrected on September 30, 2026 after independent review): three curve shapes: sharp rise then plateau (world_religions 0.32 to 0.82, high_school_biology 0.18 to 0.73), slow climb (professional_law 0.25 to 0.43), and limited improvement that stays near chance (high_school_mathematics 0.21 to 0.31, peak 0.35). Six subjects finish below 0.35, but every subject exceeds 0.25 at some checkpoint under a pointwise Wilson interval, so we do not claim that any subject "never leaves chance" until a sustained, multiplicity-adjusted definition is fixed. Seven subjects show a peak-minus-final drop strictly above 0.08 (machine_learning, abstract_algebra, global_facts, public_relations, college_mathematics, econometrics, business_ethics); because the peak is selected after inspecting the curve, these are exploratory candidates for the paired test, not evidence of forgetting. Across models on high_school_biology, Pythia-2.8B and Pythia-6.9B decline from 0.32 to about 0.25 over their runs, Amber rises from 0.18 to 0.27, and OLMo-1, OLMo-2 and K2 reach 0.65 to 0.80; the cross-model figure plots accuracy against a normalized checkpoint index (index divided by last index), which is not a

fraction of training progress where checkpoints are irregular or a release starts mid-run.

- Evaluation harness and baselines: scripts/run_baselines.py runs five trivial predictors under the walk-forward protocol at horizons 1 and 5 (results and their interpretation in Chapter 3). These results are exploratory and have already informed design; the pre-commitment is the comparison rule (beat item_mean on the same subset and horizon), and validation/test blocks will be fixed before any learner model is selected.
- Static IRT inputs: Fluid Benchmarking's fitted 2PL item parameters (a, b per item) have been loaded and inspected and are ready to initialise the temporal models.

8.5 Plan for implementing the prototype method

- Sprint 3 (Oct 1 to Oct 14): flip-rate and McNemar paired test with selection control on the seven candidate subjects; fill step and tokens_seen for Pythia, Amber and K2 from published training schedules; subject-clustered standard errors and block bootstrap in the harness; fixed chronological validation/test blocks; study of pyBKT's E-step and emIRT::dynIRT's data model; T-SKIRT with Fluid item parameters converted to the probit scale, scored at h=1 and h=5. Deliverable: first temporal model versus the item-mean bar on matching subsets.
- Sprint 4 (Oct 15 to Oct 28): static 2PL ability per checkpoint per skill as forecast (persistence) and diagnosis (held-out items) baselines; log-space batched BKT with forgetting (patched pyBKT or NumPy HMM) with unit tests for one transition per gap; DAS3H baseline; parametric-recovery simulation at N=6.
- Sprint 5 (Oct 29 to Nov 11): Martin-Quinn-style dynamic IRT with tied item parameters in NumPy (emIRT as reference), gap-scaled transition, bootstrap uncertainty; fitted-state artifact contract defined; inter-subject trajectory correlation within each run (a within-run quantity, reported alongside but not tested against the cross-model 0.965); horizons 10 and 20 evaluated; go/no-go on the dynamic IRT versus T-SKIRT as primary.
- Sprint 6 (Nov 12 to Nov 25): bank-level item-drift test with simulation-based power analysis; model-based emergence order versus threshold-based; first end-to-end figure set; Phase 2 GPU pilot and cost estimate.
- Sprint 7 (Nov 26 to Dec 12): semester-1 results write-up and figure set; visualization backend skeleton (FastAPI endpoints over the results and fitted-state Parquet defined in Sprint 5); Phase 2 go/no-go.

- Semester 2 (2027): OLMo-2 on MathCAMPS (or the no-inference fallback), deep KT ablation, GD-GPIRT on subsamples, full visualization, paper drafting, reproducibility package, NeurIPS submission (internal completion target end of April; official 2027 deadline to be confirmed, the 2026 paper deadline was May 6).

8.6 Progress log

Date	Item	Status
2026-08-31	First advisor meeting: advisor proposed project ideas; team chose knowledge tracing for LLMs across training checkpoints and refined the framing with her	Done
2026-09-01	Advisor meeting (in person): scope discussed; advisor to share relevant papers	Done
2026-09-02	Project abstract drafted and e-mailed to advisor for review	Done
2026-09-03	Abstract submitted on Canvas; GitHub repository set up; advisor feedback on abstract received by e-mail (NeurIPS target, add visualization, simplify last paragraph)	Done
2026-09-08	Team meeting: advisor feedback reviewed; knowledge-tracing reading divided (BKT; deep KT and IRT; LLM psychometrics and checkpoint releases)	Done
2026-09-15	Team meeting: paper summaries presented; time step = checkpoint raised as Decision 1; two-phase plan (MMLU subjects, then MathCAMPS)	Done
2026-09-22	First literature search (about 25 queries); closest prior work identified	Done
2026-09-23	Team meeting: literature search reviewed, core claim holds, abstract claim to soften; data verification approach agreed	Done
2026-09-29	Design decisions 1 to 5 recorded (time step = checkpoint; model family; data source; skill tags; pseudo-learners)	Done
2026-09-29	Fluid Benchmarking data verified to contain per-item, per-checkpoint correctness	Done
2026-09-29	Abstract v2 drafted per advisor feedback	Done, pending advisor review
2026-09-29	Repository skeleton, ingest pipeline (17 tests), canonical Parquet table	Done
2026-09-29	Per-skill accuracy curves with Wilson intervals; two figures	Done
2026-09-29	Evaluation harness, metrics, five baselines (33 tests); results at h=1 and h=5	Done

Date	Item	Status
2026-09-30	Deep literature review (priorities 1 to 5), spot-verified; model choice updated to dynamic IRT + BKT + T-SKIRT	Done
2026-09-30	Independent review of Workbook 1 and code: corrected empirical numbers (Section 8.4), eight bibliography records, dynamic IRT and BKT feasibility claims, baseline interpretation, Phase 2 scope; harness given a horizon argument and probability checks	Done
2026-10-14	T-SKIRT, McNemar forgetting test, clustered SEs, validation blocks	Planned

Chapter 9
Project Schedule

Two-week sprints. Owners were agreed at the team meeting of September 29, 2026; the split follows three tracks (Ronak: data and infrastructure, evaluation harness, visualization; Bhagyesh: learner models; Aniket: data analyses, statistical tests, Phase 2 evaluation) so that tasks do not overlap. Course assignments are not listed; only project outputs are. Semester-2 dates are in 2027 and assume the standard spring calendar; the NeurIPS 2027 deadline is unconfirmed and the internal completion target is the end of April. Milestones are in bold in the Deliverable column.

Sprint	Dates	Task	Owner	Deliverable
1	Aug 31 – Sep 14	Topic chosen from advisor's proposed ideas and refined with her (Aug 31, Sep 1); project framing and abstract drafted (Sep 2); repository set up (Sep 3); initial reading	Bhagyesh (background and gap); Ronak (method and evaluation, repository); Aniket (problem statement)	Project framing agreed; repository live
2	Sep 15 – Sep 30	Literature search and verification; design decisions; data verification; repo, ingest, curves, harness, baselines	Ronak (literature search, data verification, repo, ingest, harness, curves); Bhagyesh (Decisions 1–5 write-up, modeling design); Aniket (release survey, schema draft, meetings folder)	**Canonical dataset; baseline numbers; abstract v2**
3	Oct 1 – Oct 14	McNemar paired test with selection control on seven candidate subjects; fill step and tokens_seen for Pythia, Amber and K2; clustered SEs and block bootstrap; fixed validation/test blocks; pyBKT E-step and dynIRT data-model study; T-SKIRT with probit-converted item parameters	Aniket (paired test; Pythia, Amber, K2 time metadata); Ronak (clustered SEs, bootstrap, validation blocks, CI); Bhagyesh (pyBKT/dynIRT study, T-SKIRT)	**First temporal model vs item-mean bar**
4	Oct 15 – Oct 28	Static 2PL persistence and diagnosis baselines; log-space batched BKT with one-transition-per-gap tests; DAS3H baseline; recovery simulation	Bhagyesh (BKT); Aniket (static IRT baselines); Ronak (DAS3H, recovery simulation harness)	BKT results at h=1, 5

Sprint	Dates	Task	Owner	Deliverable
5	Oct 29 – Nov 11	Dynamic IRT with tied item parameters (NumPy), gap-scaled transition, bootstrap uncertainty; fitted-state artifact contract; within-run inter-subject correlation; horizons 10 and 20 evaluated; primary-model go/no-go	Bhagyesh (dynamic IRT); Aniket (correlation analysis); Ronak (horizon evaluation, fitted-state artifact contract)	**Dynamic IRT results; separability finding**
6	Nov 12 – Nov 25	Bank-level drift test with power analysis; emergence order comparison; figure set; Phase 2 GPU pilot and cost estimate	Aniket (drift test, GPU pilot); Bhagyesh (emergence analysis); Ronak (figures)	Analysis figures v1
7	Nov 26 – Dec 12	Semester-1 results write-up and figure set; FastAPI backend skeleton consuming the Sprint 5 fitted-state contract (Ronak builds; Bhagyesh takes over data endpoints in Sprint 11); Phase 2 go/no-go	Bhagyesh (model results sections); Aniket (analysis sections, go/no-go memo); Ronak (backend, figures)	**Semester-1 results write-up; backend skeleton**
8	Jan 26 – Feb 8, 2027	Phase 2 data: OLMo-2 on MathCAMPS, or DataDecide/ ElementalTask fallback with ID and coverage validation	Aniket (eval runs or fallback validation); Ronak (ingest extension); Bhagyesh (skill mapping)	Phase 2 canonical table
9	Feb 9 – Feb 22, 2027	Fit all models on Phase 2 data; deep KT ablation with leave-one-model-out on Phase 1 data (and on Phase 2 if it has more than one model); static skill-profile comparison	Bhagyesh (models); Aniket (ablation, static profiles); Ronak (harness)	Phase 2 results
10	Feb 23 – Mar 8, 2027	GD-GPIRT on subsamples; final drift and forgetting analyses; curriculum-order comparison	Aniket (GD-GPIRT, forgetting); Bhagyesh (drift, curriculum order); Ronak (results pipeline)	**Final analysis set**
11	Mar 9 – Mar 22, 2027	Visualization frontend: mastery curves with filtered/smoothed states, heatmap, forgetting markers, drift view, model comparison	Ronak (frontend); Bhagyesh (data endpoints); Aniket (user testing with advisor)	Dashboard v1

Sprint	Dates	Task	Owner	Deliverable
12	Mar 23 – Apr 5, 2027	Dashboard v2 and deployment; paper outline and related work	Ronak (deployment); Bhagyesh (method and results outline); Aniket (related work, evaluation section)	**Dashboard deployed**
13	Apr 6 – Apr 19, 2027	Paper draft; internal review; reproducibility package (pinned versions, run manifest, data-building scripts)	Bhagyesh (method, results); Aniket (introduction, related work, discussion); Ronak (figures, reproducibility package)	Paper draft v1
14	Apr 20 – May 3, 2027	Revisions from internal and advisor review; arXiv preprint	Bhagyesh (revisions to method/results); Aniket (revisions to framing); Ronak (camera-ready figures, arXiv submission)	**arXiv preprint**
15	May 4 – May 15, 2027	NeurIPS submission (date to be confirmed against the 2027 call); code and data release	Aniket (submission and checklist); Ronak (code/ data release); Bhagyesh (supplementary material)	**NeurIPS submission; public release**

9.1 Individual contributions to date

- Ronak Patel: abstract paragraph 3 (method, data, evaluation) and the abstract v2 rewrite; reading and summaries on deep knowledge tracing and its link to IRT; the systematic literature search (about 25 queries) and the deep-review brief; verification of the Fluid Benchmarking release; repository skeleton and packaging; ingest pipeline and canonical table; evaluation harness, metrics and five baselines with tests; per-skill curve figures; design-decision and project notes; Workbook 1 assembly and the corrections from its independent review.
- Bhagyesh Rathi: abstract paragraphs 1–2 (background and gap statement); reading and summaries on classic and extended Bayesian Knowledge Tracing (Corbett & Anderson; Khajah, Lindsey & Mozer; pyBKT); write-up of Design Decisions 1 and 2 (time step = checkpoint; model family given few learners) with pros and cons, and preparation of Decisions 3–5 for team adoption; advisor liaison, including the request for a recurring meeting slot; owner of the T-SKIRT implementation in Sprint 3.
- Aniket Sarrin: one-paragraph problem statement that seeded the abstract; reading and summaries on psychometrics for LLMs and public checkpoint releases (Fluid Benchmarking, MathCAMPS, Pythia, OLMo); survey of which

checkpoint releases publish item-level results; draft of the repository structure and canonical data schema; creation of the meetings folder and template; owner of the forgetting flip-rate and McNemar test in Sprint 3.